

深度学习课堂讲义

授课教师: May

2025 年 9 月 2 日

目录

1 引言	3
1.1 什么是深度学习?	3
1.2 为什么学习 CNN、RNN 和 LSTM?	3
1.3 学习目标	3
2 卷积神经网络 (CNN)	3
2.1 CNN 的基本思想	3
2.2 卷积层	4
2.3 池化层	4
2.4 参数数量计算	4
2.5 CNN 结构伪代码	4
2.6 PyTorch 实例代码	5
2.7 训练流程示例	5
2.8 超参数调优表格	6
2.9 常见应用场景	6
3 循环神经网络 (RNN)	6
3.1 基本概念	6
3.2 数学公式表示	7
3.3 梯度消失与爆炸问题	7
3.4 RNN 伪代码	7
3.5 PyTorch 实例代码	8
3.6 超参数调优表格	9
3.7 应用场景	9
3.8 课堂小结	9

4	长短期记忆网络 (LSTM)	9
4.1	提出动机	9
4.2	门控机制原理	9
4.3	公式表示	10
4.4	LSTM 伪代码	10
4.5	PyTorch 实例代码	11
4.6	超参数调优表格	12
4.7	应用场景	12
4.8	课堂小结	12

1 引言

1.1 什么是深度学习？

深度学习是机器学习的一个分支，它通过多层神经网络对数据进行逐级抽象和表示，从而自动学习到特征表示。与传统机器学习方法相比，深度学习具有以下优势：

- 自动特征提取：无需手工设计特征。
- 可扩展性强：随着数据量和计算能力提升，性能通常不断提高。
- 泛化能力强：在图像、语音、自然语言等复杂任务中表现出色。

1.2 为什么学习 CNN、RNN 和 LSTM？

1. **卷积神经网络 (CNN)**：适合图像处理、视频分析、空间结构数据建模。
2. **循环神经网络 (RNN)**：用于序列建模，能够记忆前后依赖关系。
3. **长短期记忆网络 (LSTM)**：改进了 RNN 的梯度消失问题，适合长序列任务。

1.3 学习目标

通过本课程，学生应掌握：

- CNN、RNN、LSTM 的基本原理和数学公式。
- 使用 PyTorch/TensorFlow 搭建模型的能力。
- 常见超参数及调优方法。
- 根据任务选择合适模型架构的能力。

2 卷积神经网络 (CNN)

2.1 CNN 的基本思想

卷积神经网络的设计灵感来自于人类视觉皮层结构，核心思想包括：

1. **局部感受野**：每个神经元只与输入数据的局部区域相连。
2. **权重共享**：卷积核在输入数据的不同位置滑动，参数数量大幅减少。
3. **层级结构**：低层提取边缘特征，高层提取抽象语义特征。

2.2 卷积层

二维卷积的数学公式如下：

$$y_{i,j}^{(k)} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{i+m,j+n} \cdot w_{m,n}^{(k)} + b^{(k)} \quad (1)$$

其中：

- x 表示输入特征图。
- $w^{(k)}$ 表示第 k 个卷积核。
- $b^{(k)}$ 表示偏置。
- $y^{(k)}$ 表示输出特征图。

2.3 池化层

池化层用于降低特征维度，常见方式包括最大池化和平均池化：

$$y_{i,j} = \max_{(m,n) \in R} x_{i+m,j+n} \quad (2)$$

其中 R 表示池化窗口区域。

2.4 参数数量计算

假设输入通道数为 C_{in} ，输出通道数为 C_{out} ，卷积核大小为 $K \times K$ ，则卷积层参数总数为：

$$\text{Params} = C_{out} \times (C_{in} \times K^2 + 1) \quad (3)$$

2.5 CNN 结构伪代码

Algorithm 1 CNN 前向传播伪代码

- 1: 输入图像 X
 - 2: **for** 每一层卷积层 **do**
 - 3: $X \leftarrow \text{Conv2D}(X, \text{kernel})$
 - 4: $X \leftarrow \text{ReLU}(X)$
 - 5: $X \leftarrow \text{Pooling}(X)$
 - 6: **end for**
 - 7: $X \leftarrow \text{Flatten}(X)$
 - 8: $X \leftarrow \text{FullyConnected}(X)$
 - 9: 输出预测结果
-

2.6 PyTorch 实例代码

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class SimpleCNN(nn.Module):
6     def __init__(self):
7         super(SimpleCNN, self).__init__()
8         self.conv1 = nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1)
9         self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
10        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
11        self.fc1 = nn.Linear(32 * 7 * 7, 128)
12        self.fc2 = nn.Linear(128, 10) # 假设任务是 MNIST 10分类
13
14    def forward(self, x):
15        x = self.pool(torch.relu(self.conv1(x)))
16        x = self.pool(torch.relu(self.conv2(x)))
17        x = x.view(-1, 32 * 7 * 7)
18        x = torch.relu(self.fc1(x))
19        x = self.fc2(x)
20        return x
21
22 # 初始化模型与优化器
23 model = SimpleCNN()
24 criterion = nn.CrossEntropyLoss()
25 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 1: 基于 PyTorch 的 CNN 实现

2.7 训练流程示例

```
1 for epoch in range(10):
2     running_loss = 0.0
3     for images, labels in train_loader:
4         optimizer.zero_grad()
5         outputs = model(images)
6         loss = criterion(outputs, labels)
7         loss.backward()
8         optimizer.step()
9         running_loss += loss.item()
10    print(f"Epoch {epoch+1}, Loss: {running_loss/len(train_loader)}")
```

2.8 超参数调优表格

超参数	推荐范围	注意事项
卷积核大小	$3 \times 3, 5 \times 5$	小卷积核更常用，层数可叠加
步幅 (stride)	1 或 2	控制输出尺寸，stride=2 等效于下采样
池化方式	最大池化 / 平均池化	最大池化在图像分类中更常见
学习率	$10^{-4} \sim 10^{-2}$	建议先用 Adam 优化器
Batch size	32, 64, 128	过大可能降低泛化
正则化	Dropout: 0.2 - 0.5	防止过拟合

表 1: CNN 常见超参数及调优策略

2.9 常见应用场景

- 图像分类：MNIST、CIFAR-10、ImageNet。
- 目标检测：Faster R-CNN、YOLO、SSD。
- 图像分割：U-Net、FCN。

3 循环神经网络 (RNN)

3.1 基本概念

循环神经网络 (RNN, Recurrent Neural Network) 是一类用于处理序列数据的神经网络。与前馈神经网络不同，RNN 在隐藏层中引入了循环连接，使得当前时刻的隐藏状态不仅依赖于当前输入，还依赖于上一个时刻的隐藏状态。

- 适用于时间序列数据：如语音信号、股市数据、传感器数据。
- 适用于自然语言处理：文本序列、机器翻译、语言建模。

3.2 数学公式表示

一个基本的 RNN 单元的更新公式如下：

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h) \quad (4)$$

$$y_t = W_y h_t + b_y \quad (5)$$

其中：

- x_t ：时刻 t 的输入。
- h_t ：时刻 t 的隐藏状态（包含记忆信息）。
- y_t ：时刻 t 的输出。
- W_h, W_x, W_y ：权重矩阵。
- b_h, b_y ：偏置向量。

3.3 梯度消失与爆炸问题

在训练 RNN 时，常会遇到梯度消失或梯度爆炸。考虑误差反向传播时，梯度传播公式近似为：

$$\frac{\partial L}{\partial W_h} \propto \prod_{k=1}^t \frac{\partial h_k}{\partial h_{k-1}} \quad (6)$$

如果 $\|\frac{\partial h_k}{\partial h_{k-1}}\| < 1$ ，则梯度随时间衰减，导致**梯度消失**；如果 $\|\frac{\partial h_k}{\partial h_{k-1}}\| > 1$ ，则梯度随时间指数增长，导致**梯度爆炸**。

3.4 RNN 伪代码

Algorithm 2 RNN 前向传播伪代码

- 1: 初始化隐藏状态 h_0
 - 2: **for** $t = 1$ 到 T **do**
 - 3: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$
 - 4: $y_t = W_y h_t + b_y$
 - 5: **end for**
 - 6: 输出 $\{y_1, y_2, \dots, y_T\}$
-

3.5 PyTorch 实例代码

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # 定义简单 RNN 模型
6 class SimpleRNN(nn.Module):
7     def __init__(self, input_size, hidden_size, output_size, num_layers=1):
8         super(SimpleRNN, self).__init__()
9         self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True
10                             )
11         self.fc = nn.Linear(hidden_size, output_size)
12
13     def forward(self, x):
14         out, h_n = self.rnn(x) # out: (batch, seq_len, hidden_size)
15         out = self.fc(out[:, -1, :]) # 取最后时刻的隐藏状态
16         return out
17
18 # 初始化模型与训练组件
19 model = SimpleRNN(input_size=10, hidden_size=50, output_size=2, num_layers=1)
20 criterion = nn.CrossEntropyLoss()
21 optimizer = optim.Adam(model.parameters(), lr=0.001)
22
23 # 训练循环示例
24 for epoch in range(5):
25     for seq, labels in train_loader:
26         optimizer.zero_grad()
27         outputs = model(seq)
28         loss = criterion(outputs, labels)
29         loss.backward()
30         optimizer.step()
31     print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Listing 3: RNN 模型定义与训练

3.6 超参数调优表格

超参数	推荐范围	说明与注意事项
隐藏层维度	50 – 200	较大维度提高表达能力，但可能过拟合
层数	1 – 3	增加层数提高性能，但训练更困难
学习率	$10^{-4} \sim 10^{-2}$	Adam 优化器对学习率更稳定
序列长度	20 – 100	过长序列易导致梯度消失/爆炸
梯度裁剪 (clip)	1 – 5	防止梯度爆炸的常用技巧
Dropout	0.2 – 0.5	防止过拟合

表 2: RNN 常见超参数及调优建议

3.7 应用场景

- **语音识别**：输入为语音序列，输出为文字。
- **文本分类**：输入为句子，输出为情感标签。
- **时间序列预测**：输入为历史数据，输出为未来趋势。

3.8 课堂小结

- RNN 能够处理序列数据，但容易出现梯度消失或爆炸。
- 超参数如隐藏层维度、序列长度、梯度裁剪对性能有较大影响。
- 为了克服长程依赖问题，引入了改进模型——LSTM。

4 长短期记忆网络 (LSTM)

4.1 提出动机

在基本 RNN 中，随着序列长度增加，模型往往出现梯度消失或爆炸，导致难以学习长期依赖关系。LSTM (Long Short-Term Memory, 长短期记忆网络) 通过引入门控机制，有效地解决了这一问题。

4.2 门控机制原理

LSTM 单元包含三个门（遗忘门、输入门、输出门）以及一个细胞状态 (cell state)。其核心思想是：

- **遗忘门** f_t : 决定遗忘多少过去的信息。
- **输入门** i_t : 决定当前输入的信息有多少被写入记忆。
- **候选记忆** $\tilde{C}_t$: 对当前输入生成候选信息。
- **细胞状态** C_t : 贯穿整个序列的“记忆通道”，用于长期依赖。
- **输出门** o_t : 决定输出多少信息给隐藏状态。

4.3 公式表示

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{遗忘门}) \quad (7)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{输入门}) \quad (8)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{候选记忆}) \quad (9)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (\text{细胞状态更新}) \quad (10)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{输出门}) \quad (11)$$

$$h_t = o_t * \tanh(C_t) \quad (\text{隐藏状态更新}) \quad (12)$$

其中 σ 表示 sigmoid 函数， $*$ 表示逐元素乘法。

4.4 LSTM 伪代码

Algorithm 3 LSTM 前向传播伪代码

```

1: 初始化  $h_0, C_0$ 
2: for  $t = 1$  到  $T$  do
3:    $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ 
4:    $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ 
5:    $\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ 
6:    $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$ 
7:    $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ 
8:    $h_t = o_t * \tanh(C_t)$ 
9: end for
10: 输出  $\{h_1, h_2, \dots, h_T\}$ 

```

4.5 PyTorch 实例代码

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class SimpleLSTM(nn.Module):
6     def __init__(self, input_size, hidden_size, output_size, num_layers=1):
7         super(SimpleLSTM, self).__init__()
8         self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=
            True)
9         self.fc = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         out, (h_n, c_n) = self.lstm(x) # out: (batch, seq_len, hidden_size)
13         out = self.fc(out[:, -1, :]) # 取最后时刻的输出
14         return out
15
16 # 初始化模型与优化器
17 model = SimpleLSTM(input_size=10, hidden_size=100, output_size=2, num_layers=2)
18 criterion = nn.CrossEntropyLoss()
19 optimizer = optim.Adam(model.parameters(), lr=0.001)
20
21 # 训练循环
22 for epoch in range(5):
23     for seq, labels in train_loader:
24         optimizer.zero_grad()
25         outputs = model(seq)
26         loss = criterion(outputs, labels)
27         loss.backward()
28         nn.utils.clip_grad_norm_(model.parameters(), max_norm=5) # 防止梯度爆
            炸
29         optimizer.step()
30     print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Listing 4: PyTorch LSTM 实现

4.6 超参数调优表格

超参数	推荐范围	说明与注意事项
隐藏层维度	100 – 300	更大维度适合复杂任务
层数 (num_layers)	1 – 3	过深可能导致梯度难以传播
学习率	$10^{-4} \sim 10^{-2}$	建议配合 Adam
序列长度	50 – 200	LSTM 能处理更长依赖，但太长仍困难
梯度裁剪 (clip)	1 – 5	减少梯度爆炸风险
Dropout	0.2 – 0.5	防止过拟合
Batch size	32 – 128	与内存和任务规模相关

表 3: LSTM 常见超参数及调优建议

4.7 应用场景

- **自然语言处理**：机器翻译、文本生成、问答系统。
- **语音处理**：语音识别、语音合成。
- **时间序列建模**：股票预测、传感器数据分析。

4.8 课堂小结

- LSTM 通过门控机制解决了 RNN 的梯度消失问题。
- 细胞状态 C_t 是记忆的核心，可以长时间存储信息。
- 实践中需注意梯度裁剪、Dropout 等技巧以提升稳定性。